

Week 2

2. Graphical User Interface Design

Creating a GUI with Java Swing

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, January 14th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda

- Creating GUIs with Java Swing
 - Based on Oracle Swing Tutorial
- Step by step presentation
- Resources
- Summary



Part I

About Java Swing



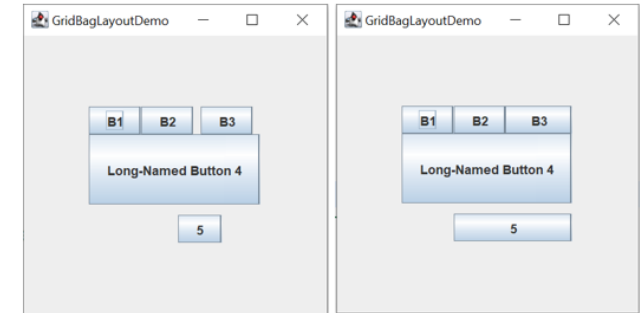


About Java Swing



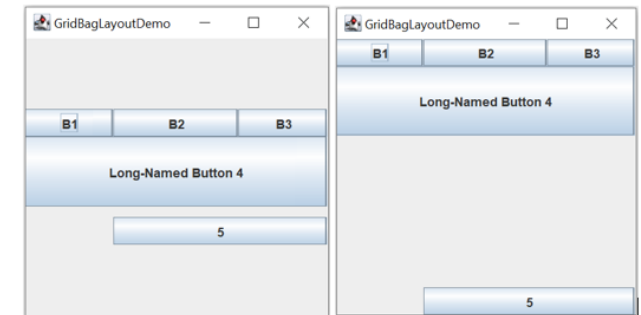
What is Java Swing?

- Part of Java Foundation Classes (JFC)
- Lightweight components
- Platform independent



(a) Natural width and height

(b) fill = HORIZONTAL



(c)

Column 1: weightx = 0.5
Column 2: weightx = 1.0
Column 3: weightx = 0.5

(d)

Row 1: weighty = 0 (not participating)
Row 2: weighty = 0 (not participating)
Row 3: weighty = 1.0 (take all)

Swing vs AWT

- Abstract Window Toolkit (AWT): Heavyweight, OS-dependent
- Swing: Lightweight, pure Java
- Pluggable Look & Feel
- Better Consistency across various OSs

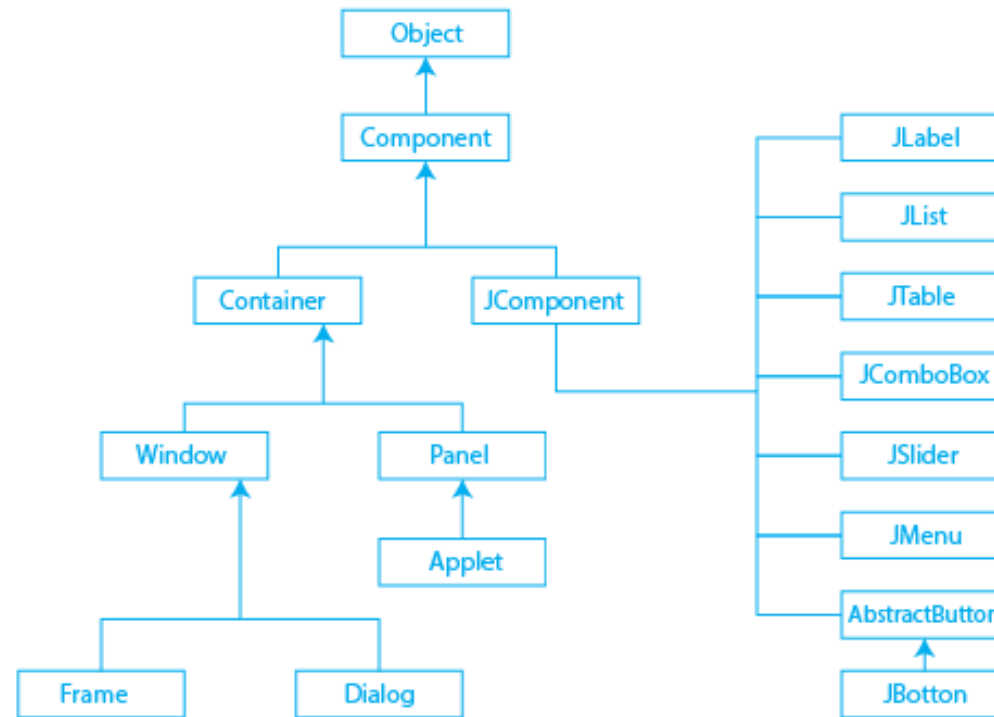
Java AWT	Java Swing
AWT components are platform-dependent .	Java swing components are platform-independent .
AWT components are heavyweight .	Swing components are lightweight .
AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
AWT doesn't follow MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC .

Reference: <https://www.geeksforgeeks.org/java/difference-between-awt-and-swing-in-java/>



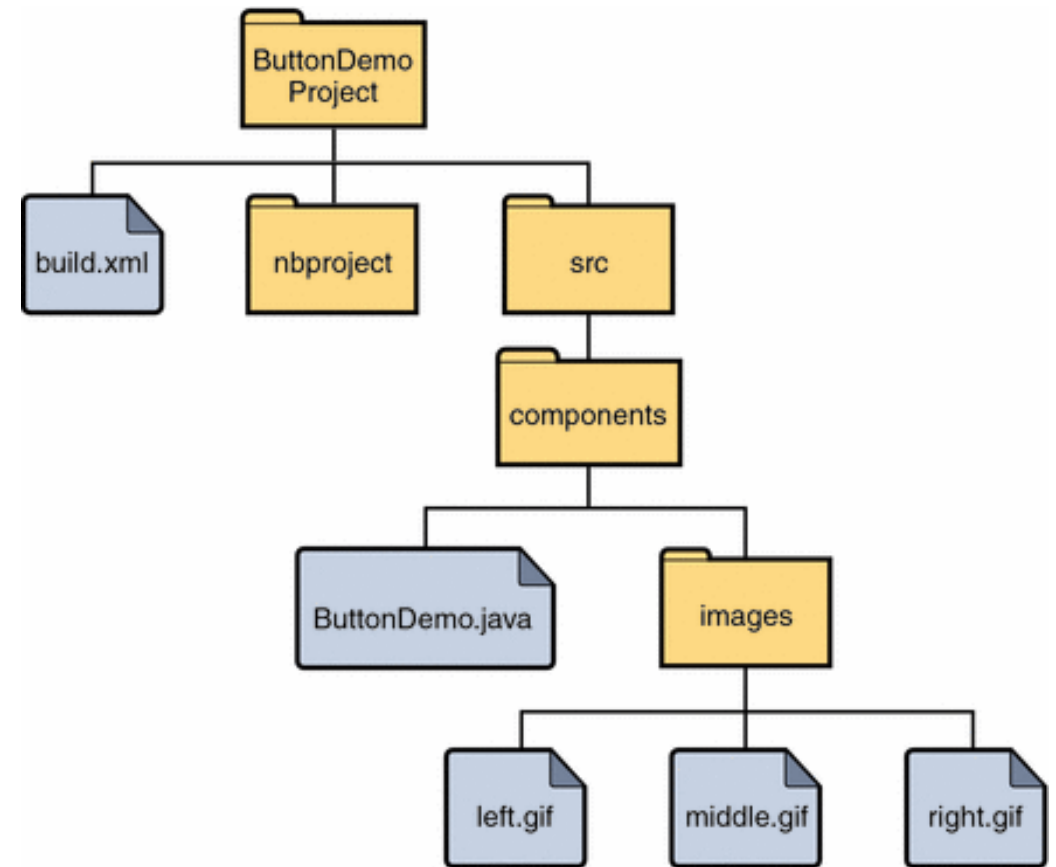
Swing Application Structure

- JFrame (window)
- JPanel (containers)
- Components
- Event handling



Core Swing Packages

- `javax.swing`
- `java.awt`
- `java.awt.event`



Part II

Java Swing details



Java Swing: The first program



Your First Swing Program

- Create JFrame
- Set size
- Set close operation
- Make visible

```
import javax.swing.*;

public class HelloSwing {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Hello Swing");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

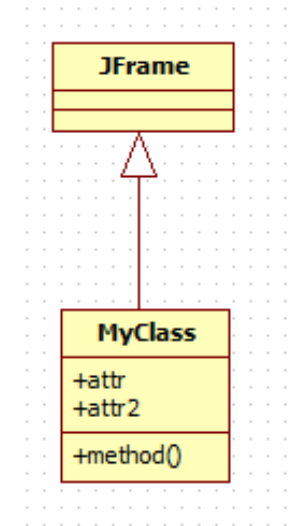
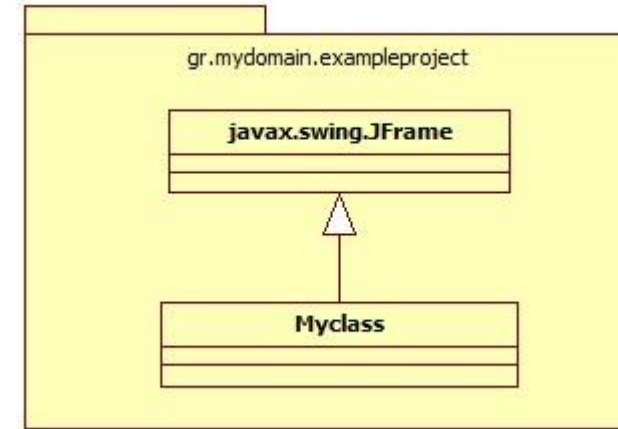


Java Swing: Elements



JFrame Explained

- Top-level window
- Manages content pane
- Handles close operations



Adding Components

- `frame.add(component)`
- Uses content pane

```
// Source - https://stackoverflow.com/q
// Posted by Haxed, modified by community. See post 'Timeline' for change history
// Retrieved 2026-01-14, License - CC BY-SA 4.0
```

```
import java.awt.Color;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
```

```
public class Control extends JFrame {

    // JPanel
    JPanel pnlButton = new JPanel();
    // Buttons
    JButton btnAddFlight = new JButton("Add Flight");

    public Control() {
        // FlightInfo setbounds
        btnAddFlight.setBounds(60, 400, 220, 30);
```

```
        // JPanel bounds
        pnlButton.setBounds(800, 800, 200, 100);
```



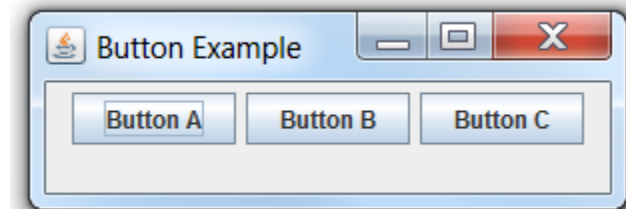
Buttons with JButton

- User interaction
- Triggers events

```
import java.awt.Container;
import javax.swing.JFrame;

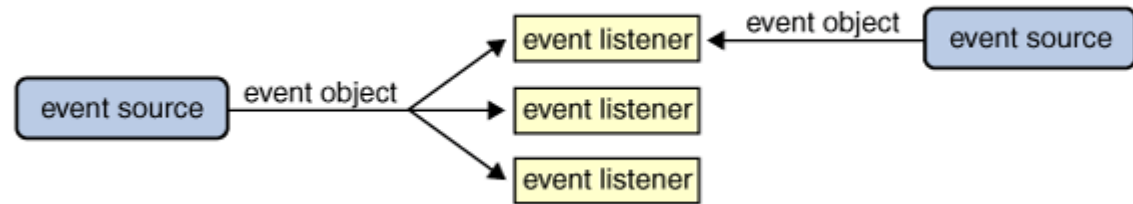
public class ButtonFrameTest {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        ButtonFrame frame = new ButtonFrame();
        frame.pack();
        frame.setVisible(true);
    }
}
```



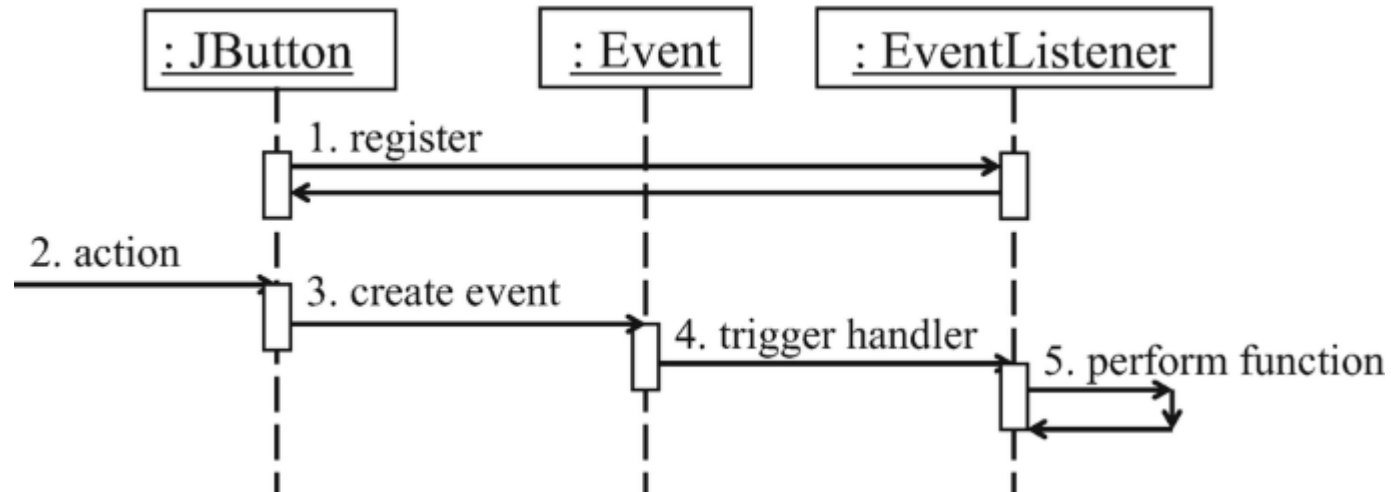
Event Handling Model

- Events
- Listeners
- Callbacks



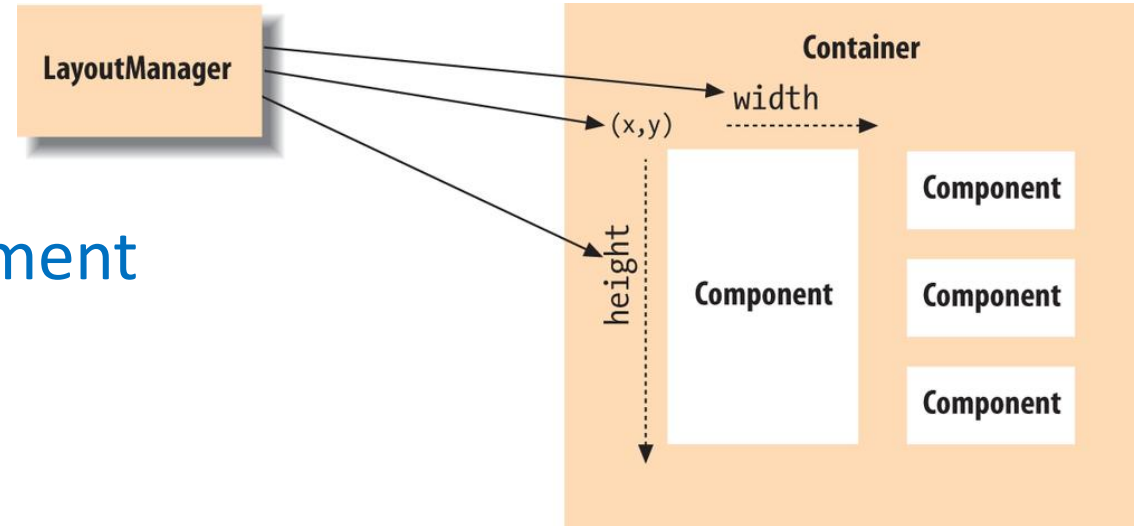
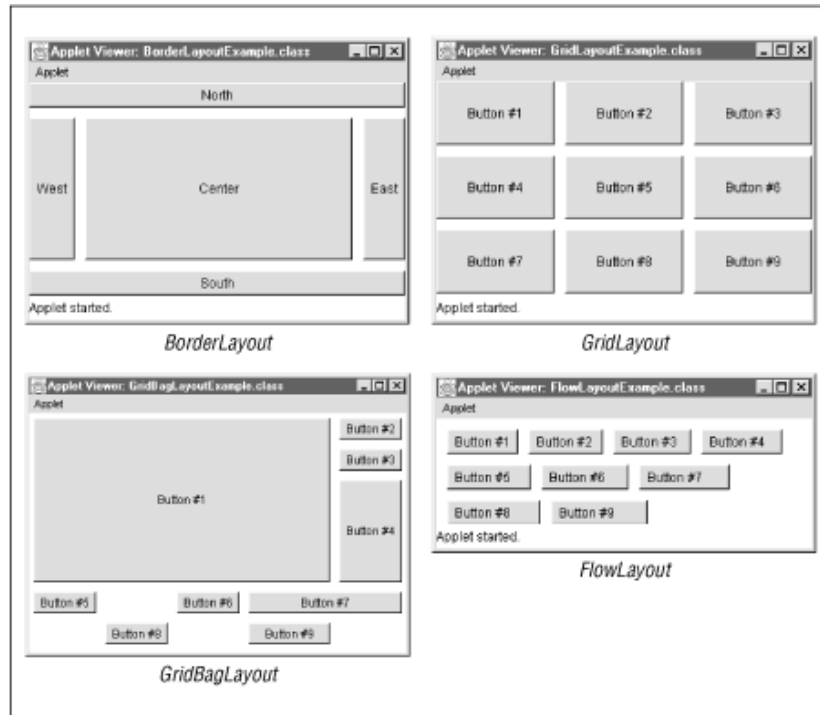
ActionListener Example

- Responds to button clicks



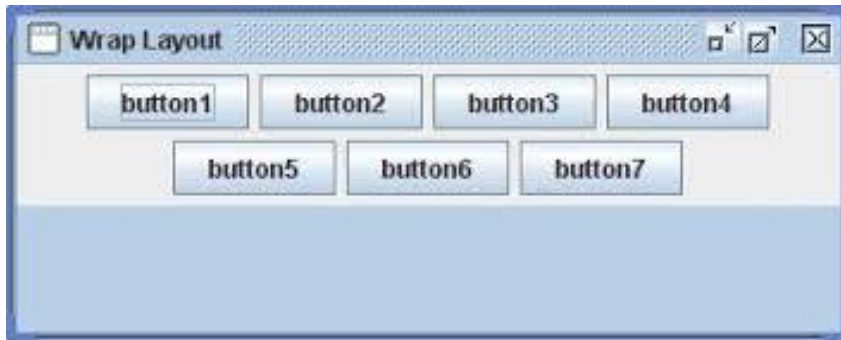
Layout Managers

- Automatic component arrangement
- Avoid absolute positioning



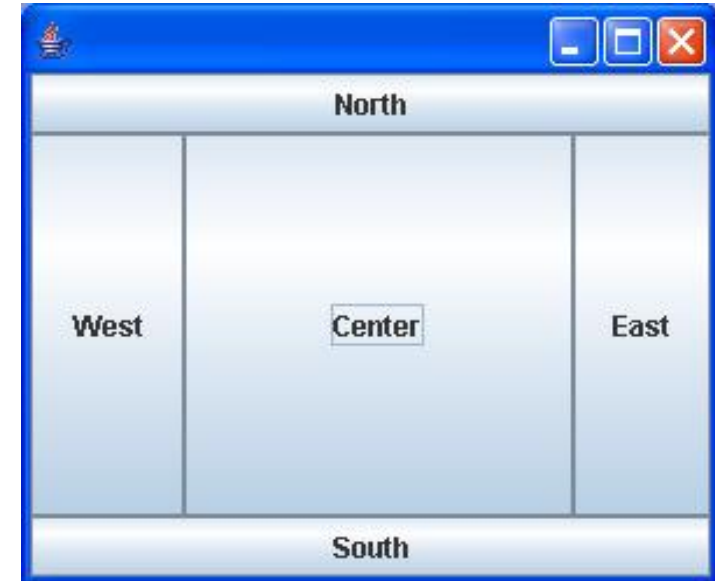
FlowLayout

- Left-to-right flow
- Wraps components



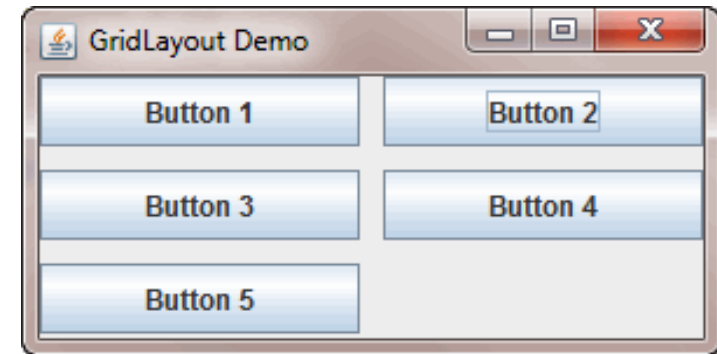
BorderLayout

- North, South, East, West, Center



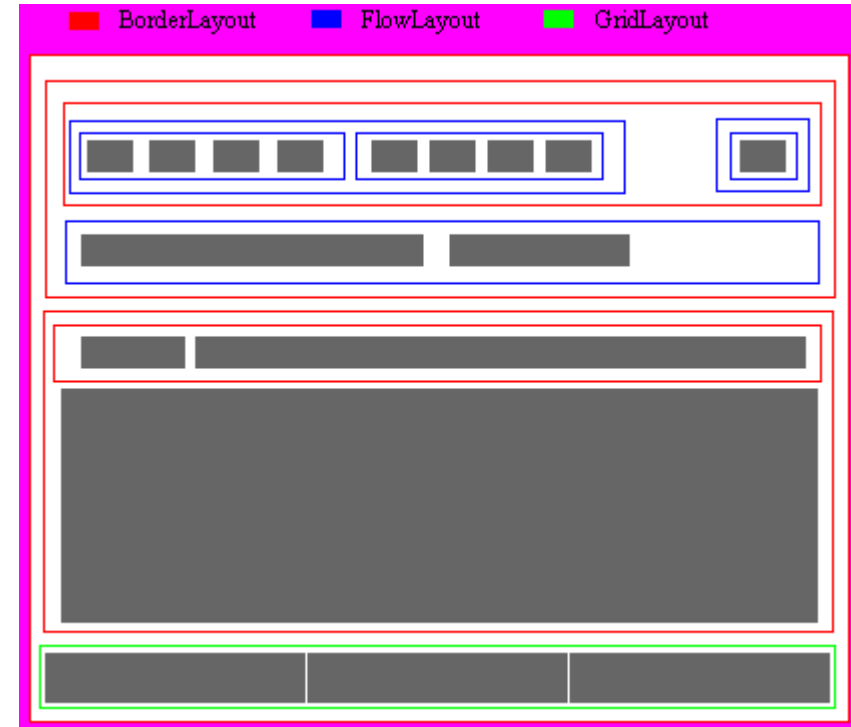
GridLayout

- Rows and columns
- Equal sizes



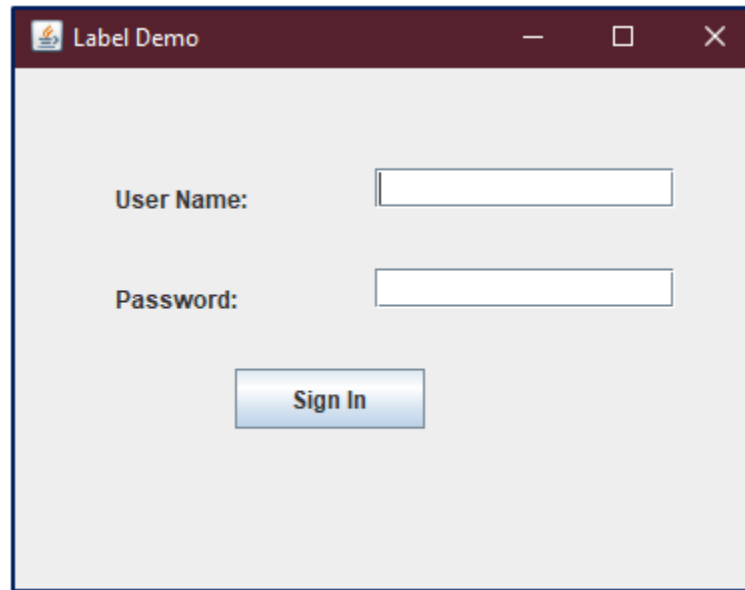
Using JPanel

- Group components
- Nest layouts



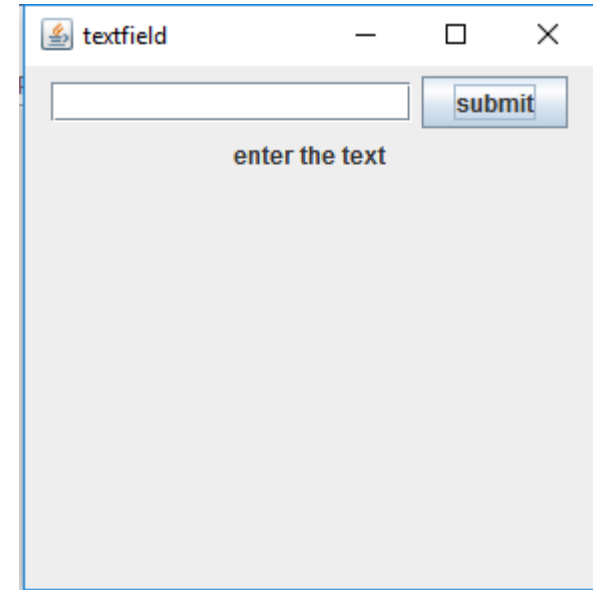
JLabel

- Displays text or images



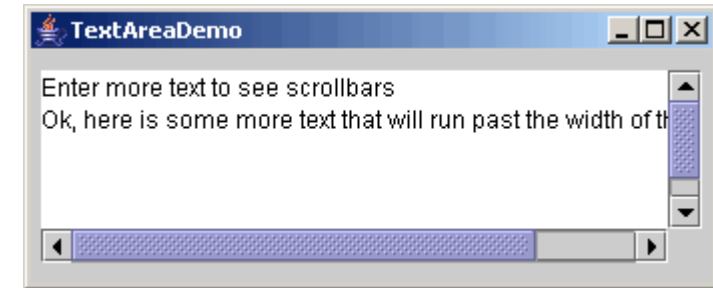
● JTextField

- • Single-line user input



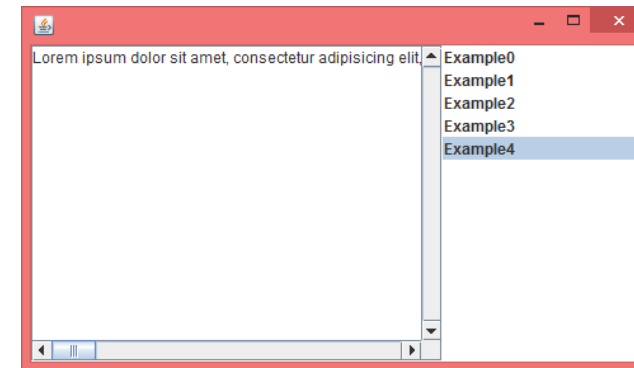
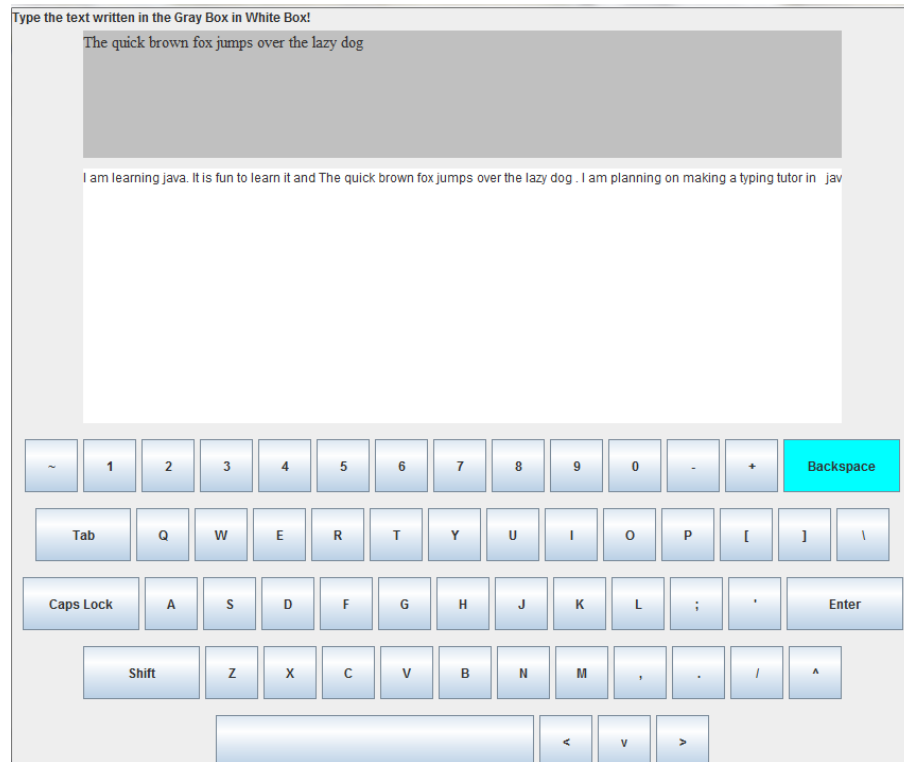
● JTextArea

- Multi-line text
- Scrollable

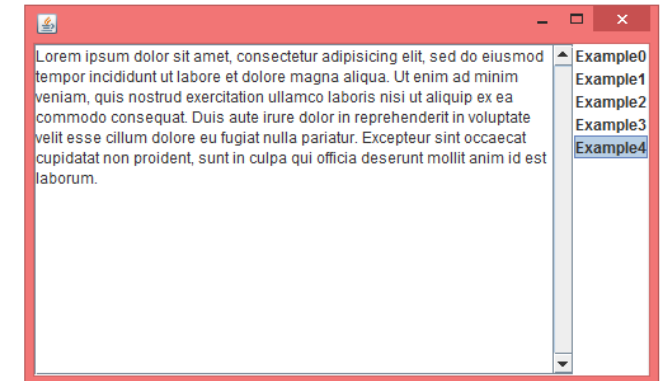


Scroll Panes

- Automatic scrolling



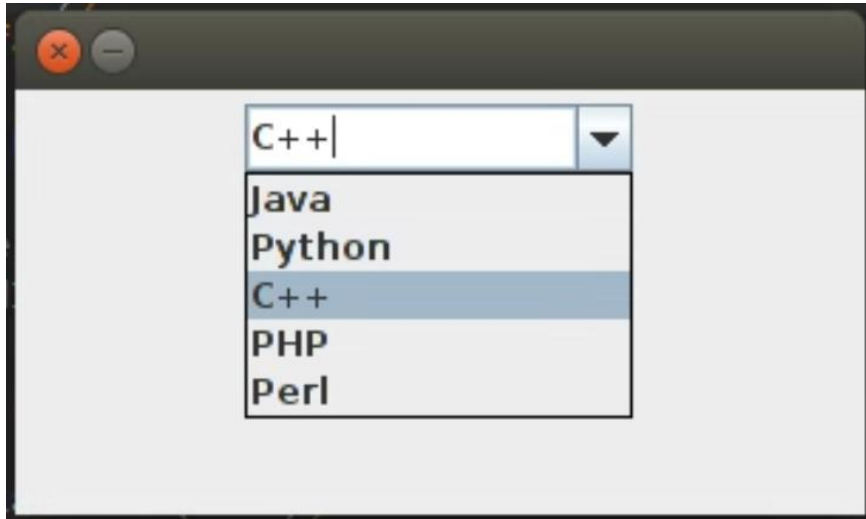
No wrapping



Wrapping

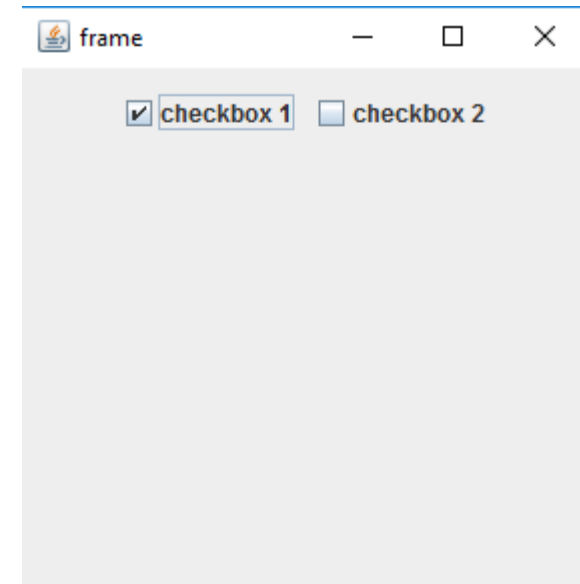
JComboBox

- Drop-down selection



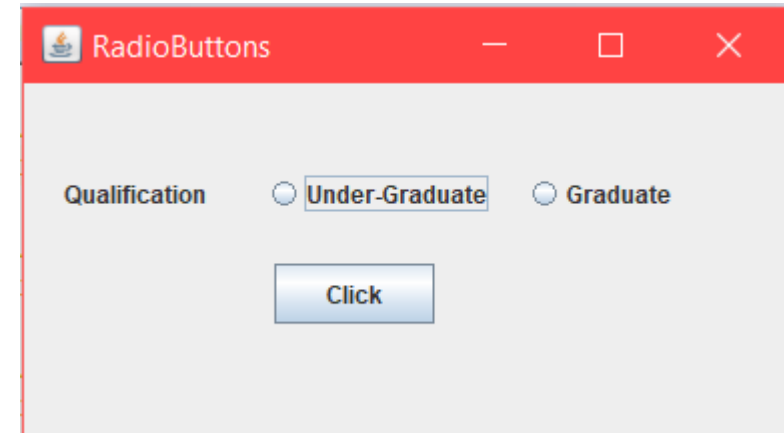
JCheckBox

- Multiple options



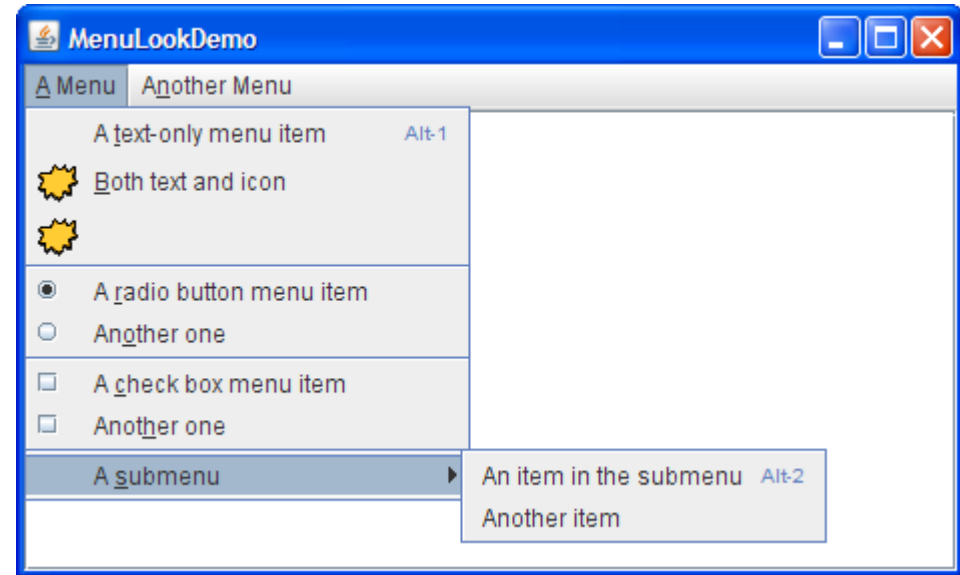
JRadioButton

- Exclusive selection
- ButtonGroup



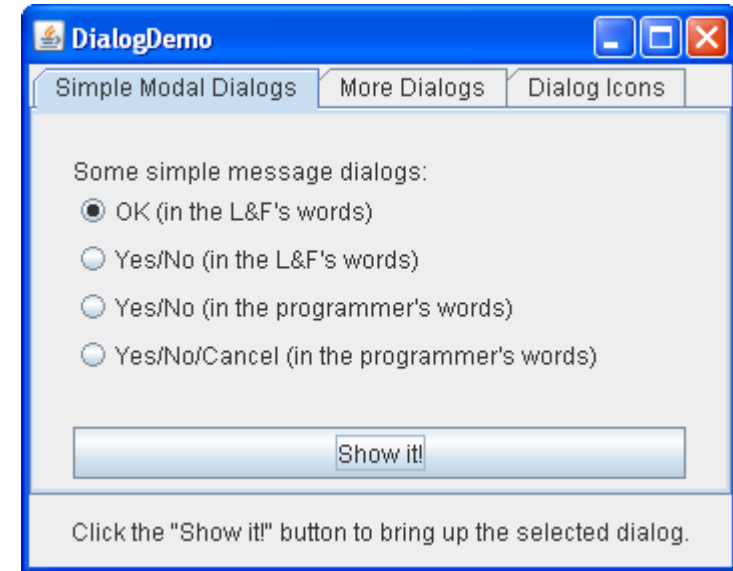
Menus

- JMenuBar
- Jmenu
- JMenuItem



Dialogs

- JOptionPane

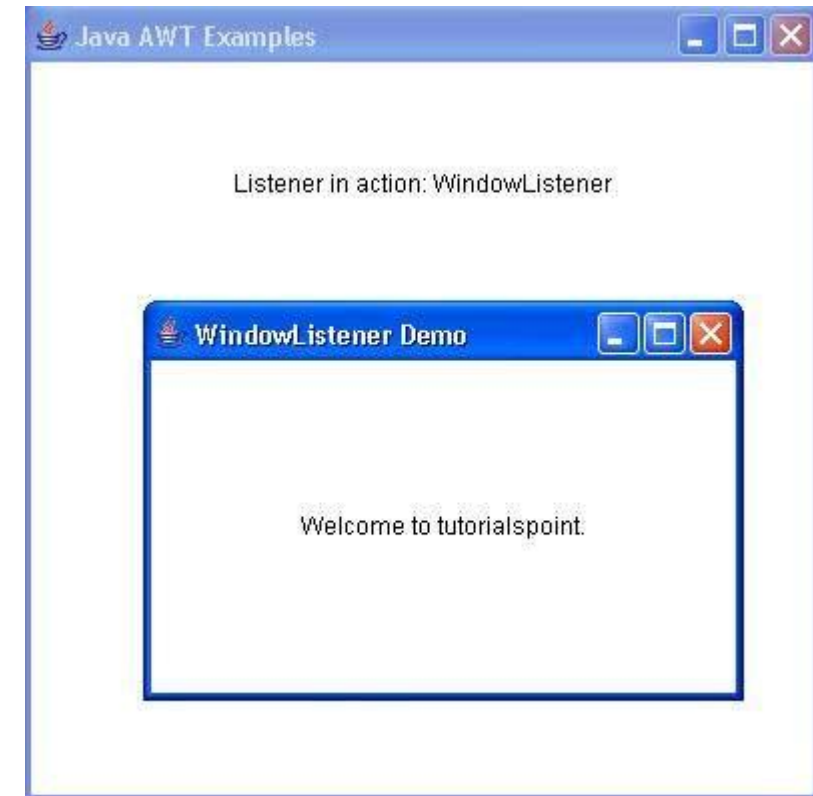


Window Events

- WindowListener

- [Image Placeholder]

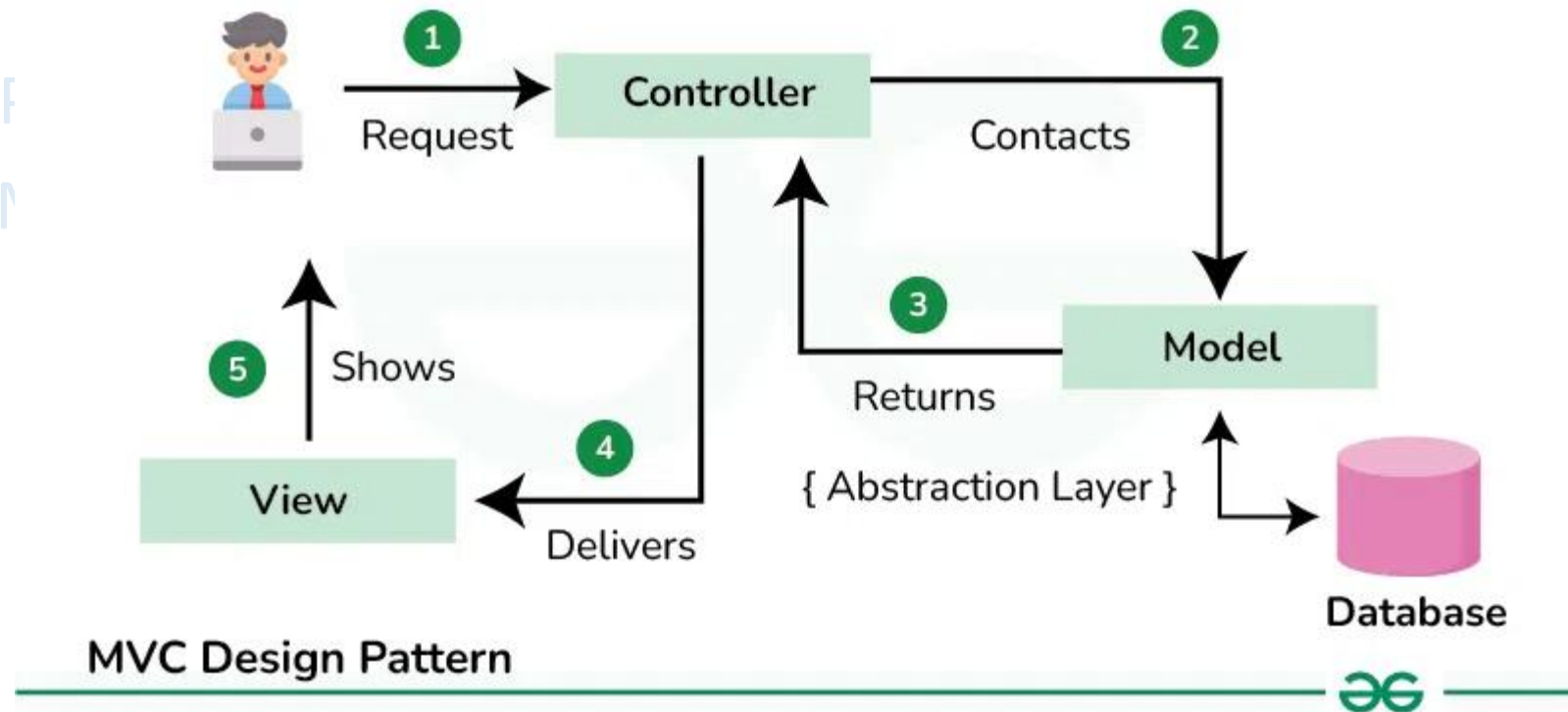
- Image: Window lifecycle diagram



MVC Pattern in Swing

- Model – View – Controller (MVC)

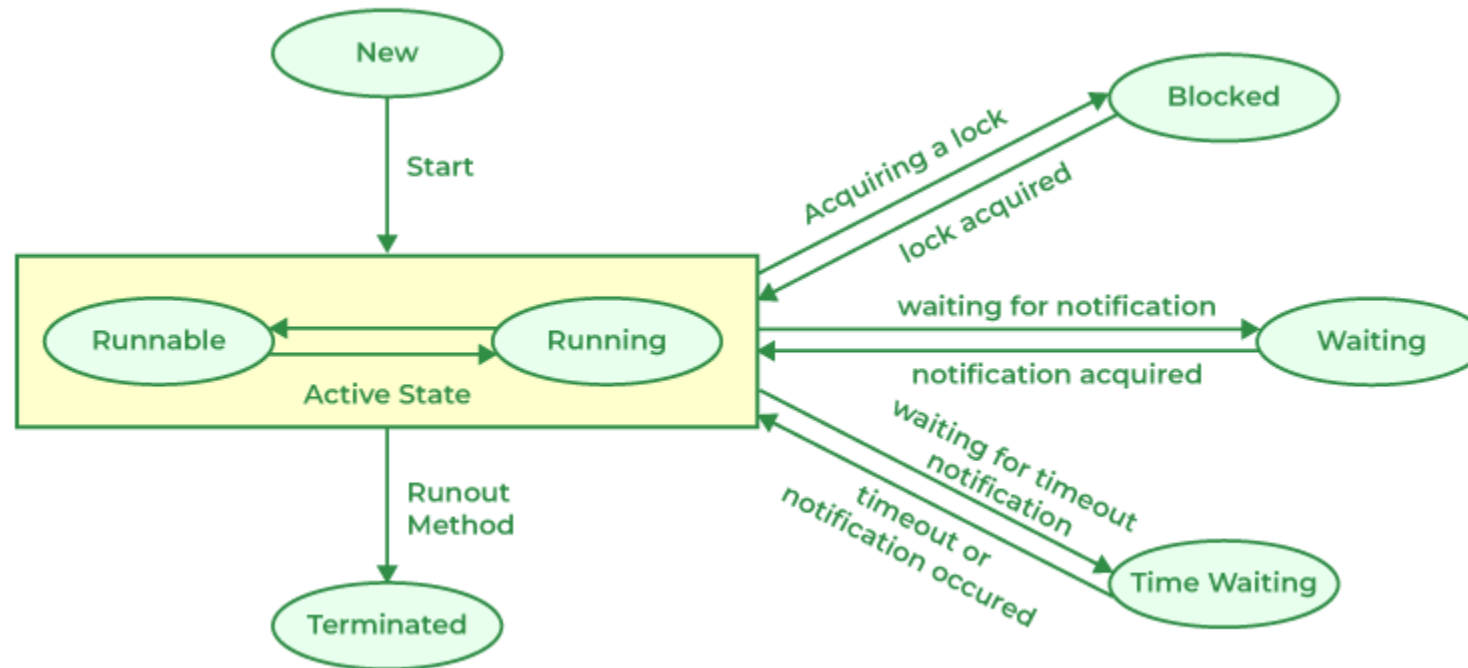
- [Image f
- Image: f



Swing & Threads

- Event Dispatch Thread (EDT)

- [Imag
- Image

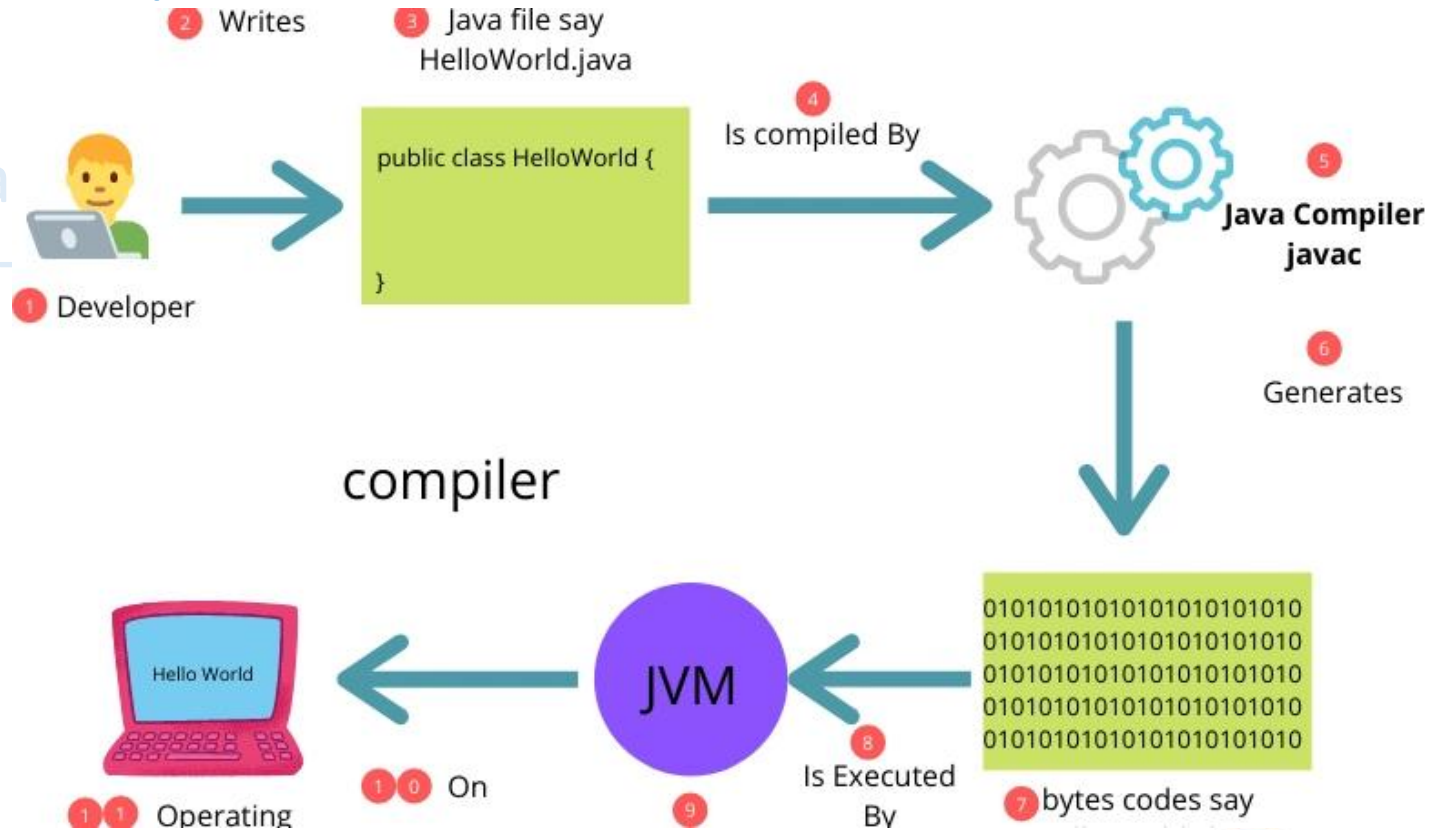


● invokeLater()

○ • Safe GUI startup

• [Image Pla

• Image: ED

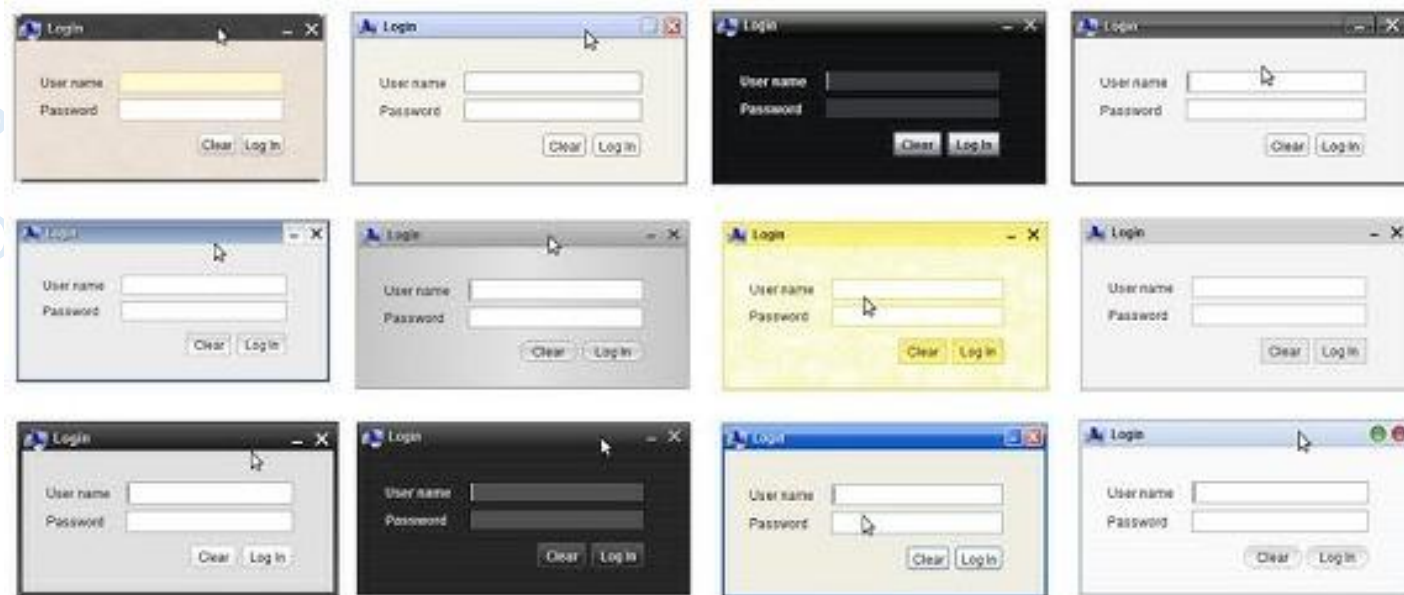


Look and Feel

- System or custom styles

• [Image Placeholder]

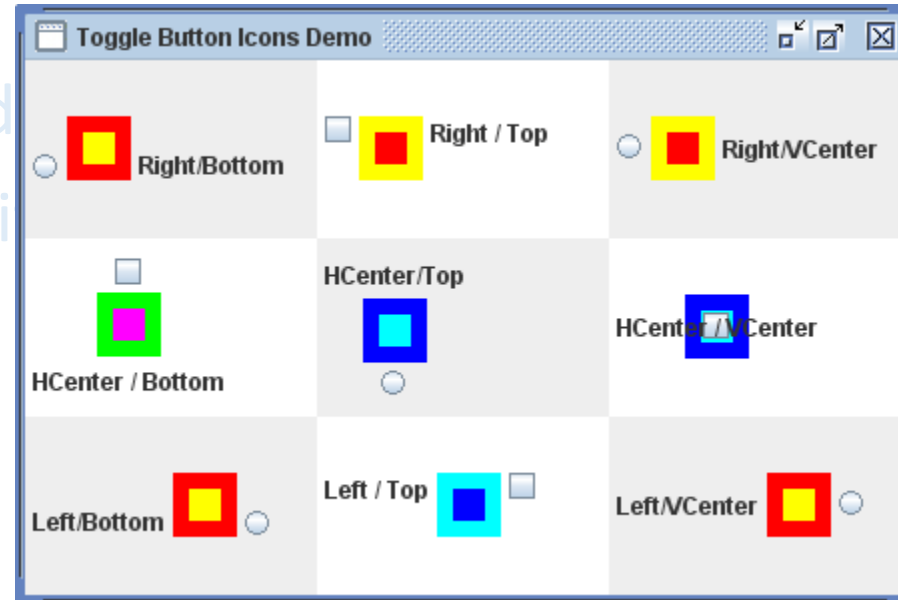
• Image: D



Images & Icons

- ImageIcon usage

- [Image Placeholder]
- Image: Button with

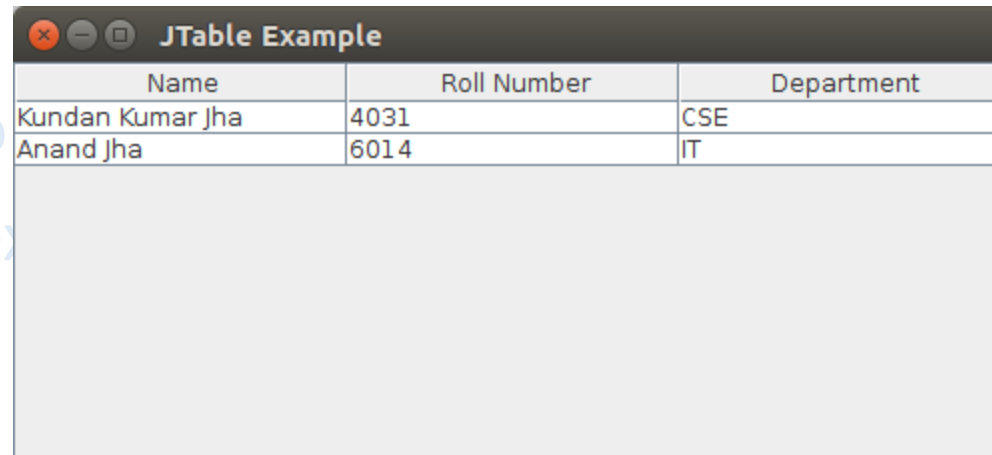


● JTable

- • Tabular data display

• [Image Placeholder]

• Image: JTable example

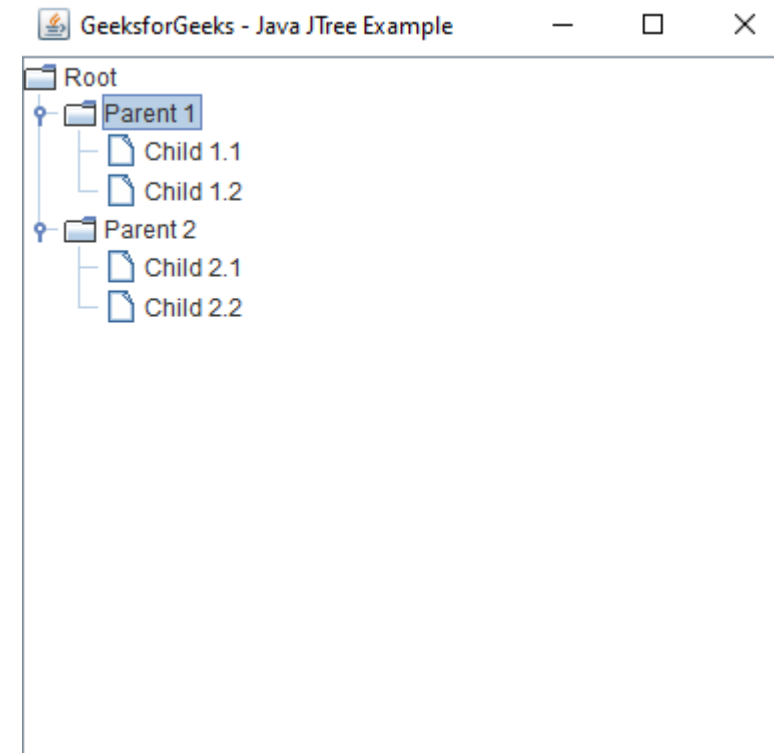


A screenshot of a Java Swing window titled "JTable Example". The window contains a table with three columns: "Name", "Roll Number", and "Department". The table has two data rows: "Kundan Kumar Jha" with Roll Number "4031" and Department "CSE", and "Anand Jha" with Roll Number "6014" and Department "IT". The table is displayed within a standard Java Swing frame with a title bar and window controls.

Name	Roll Number	Department
Kundan Kumar Jha	4031	CSE
Anand Jha	6014	IT

● JList & JTree

- • Lists and hierarchies
- [Image Placeholder]
- Image: JTree file explorer



Best Practices

- Layout managers
- MVC
- EDT safety

- [Image Placeholder]
- Image: Checklist graphi

Java Code Review Checklist	
Code Style and Formatting ☐ Consistent indentation and formatting ☐ Proper naming conventions ☐ Consistent use of braces	Code Quality ☐ No code duplication (DRY principle) ☐ Proper exception handling ☐ Clear and concise method names ☐ SOLID principles applied ☐ Efficient use of data structures
Documentation ☐ Javadoc for public methods and classes ☐ Inline comments for complex logic ☐ README and other necessary documentation	Testing ☐ Unit tests for new code ☐ Integration tests where necessary ☐ Edge cases and error scenarios covered

Summary



● Summary



- Swing components
- Events
- Layouts
- [Image Placeholder]
- Image: Concept map





Resources



Resources

- [Oracle Swing Tutorial](#)
- [JavaDocs](#)
- [Introduction to Java Swing \[GfG\]](#)
- [Swing Tutorial \[TutorialsPoint\]](#)
- Textbook (Chapter 10)
- [Image Placeholder]
- Image: Oracle documentation screenshot



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor